

LAPORAN QUIZ PRAKTIKUM 11: LOGISTIC REGRESSION

Prediksi Tingkat Stres Mahasiswa Berdasarkan Beban Tugas

Nama : Ruud Zaki Bramdani

NIM : 2024081028

Prodi : SIF

1. Identifikasi Komponen Klasifikasi

Berdasarkan data dari spreadsheet survei yang digunakan, berikut adalah komponen variabel yang diidentifikasi untuk pemodelan machine learning:

- **Objek:** Mahasiswa yang mengisi survei beban tugas dan tingkat stres.
- **Fitur (X):** 1. Jumlah tugas (Kuantitas tugas saat ini) 2. Jumlah tugas melewati deadline (Kuantitas tugas yang terlambat) 3. Rata-rata jam tidur (Durasi istirahat dalam jam)
- **Target / Label (y):** Status stress (0 = Tidak Stres, 1 = Stres)

2. Tahap Data Cleaning dan Preprocessing

Sebelum dimasukkan ke dalam model Logistic Regression, data mentah dari spreadsheet dibersihkan terlebih dahulu melalui langkah-langkah berikut agar hasilnya akurat:

1. **Handling Missing Values & Duplicates:** Menghapus baris data yang kosong atau baris yang terduplikasi secara tidak sengaja saat pengisian form.
2. **Standardisasi Format:** Memastikan kolom NIM, Jumlah Tugas, SKS, dan Jam Tidur bertipe numerik yang konsisten.
3. **Label Encoding:** Mengubah teks pada kolom target Status stress menjadi angka biner:
 - Teks "**Stres**" diubah menjadi angka **1**
 - Teks "**Tidak Stres**" diubah menjadi angka **0**

3. Source Code Implementasi

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Membaca dataset langsung dari file yang diupload di Google Colab
# Pastikan nama file sesuai dengan file spreadsheet survei yang kamu miliki
try:
    df = pd.read_excel("/content/Survei Beban Tugas & Tingkat Stres Mahasiswa (Jawaban).xlsx")
    print("Dataset berhasil dimuat!")
except Exception as e:
    print(f"File tidak ditemukan atau terjadi kesalahan: {e}")
    print("Pastikan file sudah di-upload ke Google Colab dan nama file benar.")

# 2. Data Cleaning & Preprocessing
print(f"Initial DataFrame shape: {df.shape}")
print("Missing values before cleaning:")
print(df.isnull().sum())

# Drop 'Yes' column as it has all missing values and is not used for analysis
if 'Yes' in df.columns:
    df.drop('Yes', axis=1, inplace=True)
    print("Dropped 'Yes' column.")

# Rename columns for easier access
df.rename(columns={
    'Saat ini, ada berapa total tugas kuliah kamu yang masih aktif atau belum selesai?': 'Jumlah tugas',
    'Hari tugas-tugas tersebut, berapa jumlah tugas yang deadline-nya mepet (kurang dari 3 hari)': 'Jumlah tugas melewati deadline',
    'Berapa rata-rata jam tidur kamu per hari dalam seminggu terakhir?': 'Rata-rata jam tidur',
    'Secara keseluruhan, apakah kamu merasa stres dengan beban akademisku saat ini?': 'Status stress'
}, inplace=True)

# Now apply dropna and drop_duplicates after cleaning the problematic 'Yes' column and renaming.
df.dropna(inplace=True) # Hapus missing values
df.drop_duplicates(inplace=True) # Hapus data duplikat

print(f"DataFrame shape after cleaning and renaming: {df.shape}")

# Check if dataframe is empty after cleaning
if df.empty:
    print("DataFrame is empty after cleaning. Please check your data for excessive missing values or duplicates.")
else:
    print(f"Unique values in 'Status stress' before mapping:")
    print(df['Status stress'].unique())

# Ensure 'Status stress' column is string type and strip whitespace
df['Status stress'] = df['Status stress'].astype(str).str.strip()

# Mapping label target (Tekn ke Biner)
# Corrected mapping based on unique values observed: ['Ya', 'Tidak', 'Stress']
df['Status stress'] = df['Status stress'].map({'Stres': 1, 'Tidak stres': 0, 'Ya': 1, 'Tidak': 0, 'Stress': 1})

print("Unique values in 'Status stress' after mapping:")
print(df['Status stress'].unique())

# Drop rows where 'Status stress' became NaN after mapping (if any)
if df['Status stress'].isnull().any():
    initial_rows = len(df)
    df.dropna(subset=['Status stress'], inplace=True)
    print(f"Dropped {initial_rows - len(df)} rows with NaN in 'Status stress' after mapping.")

# 3. Menentukan Fitur (X) dan Target (y)
X = df[['Jumlah tugas', 'Jumlah tugas melewati deadline', 'Rata-rata jam tidur']]
y = df['Status stress']

# Check if X and y are empty before splitting
if X.empty or y.empty:
    print("Features (X) or target (y) are empty after column selection. Please check column names and data.")
else:
    # 4. Membagi data menjadi 80% Training dan 20% Testing
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # 5. Membangun dan Melatih Model Logistic Regression
    model = LogisticRegression()
    model.fit(X_train, y_train)

    # 6. Melakukan Prediksi Klasifikasi dan Probabilitas
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)

    # 7. Menampilkan Hasil Evaluasi Model
    print("\n=== EVALUASI MODEL REGRESI LOGISTIK ===")
    print(f"Akurasi Model: {accuracy_score(y_test, y_pred) * 100:.2f}%")

    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    print("\n=== CONTOH PROBABILITAS PREDIKSI (5 DATA PERTAMA UJI) ===")
    for i in range(min(5, len(X_test))):
        print(f"Data ke-{i+1} -> Probabilitas [Tidak Stres, Stres]: {y_prob[i]} -> Prediksi: {y_pred[i]}")
```

4. Hasil Output Pengujian

```
Dataset berhasil dimuat!
Initial DataFrame shape: (40, 6)
Missing values before cleaning:
timestamp      3
Nama lengkap   3
Nilai          3
Saat ini, ada berapa total tugas kuliah kamu yang masih aktif atau belum selesai?  3
Hari tugas-tugas tersebut, berapa jumlah tugas yang deadline-nya mepet (kurang dari 3 hari)?  3
Berapa rata-rata jam tidur kamu per hari dalam seminggu terakhir?  3
Secara keseluruhan, apakah kamu merasa stres dengan beban akademisku saat ini?  3
Yes            40
dtype: object
Dropped 'Yes' column.
DataFrame shape after cleaning and renaming: (37, 7)
Unique values in 'Status stress' before mapping:
['Ya', 'Tidak', 'Stress']
Unique values in 'Status stress' after mapping:
[1, 0]
=== EVALUASI MODEL REGRESI LOGISTIK ===
Akurasi Model: 75.86%
Confusion Matrix:
[[1 0]
 [2 5]]
Classification Report:
              precision    recall  f1-score   support

0               0.33         1.00         0.50         1
1               1.00         0.71         0.83         7

accuracy               0.75         0.75         0.75         8
macro avg              0.67         0.86         0.67         8
weighted avg           0.92         0.75         0.79         8

=== CONTOH PROBABILITAS PREDIKSI (5 DATA PERTAMA UJI) ===
Data ke-1 -> Probabilitas [Tidak Stres, Stres]: [0.7979202 0.2420798] -> Prediksi: 0
Data ke-2 -> Probabilitas [Tidak Stres, Stres]: [0.62832073 0.37167927] -> Prediksi: 0
Data ke-3 -> Probabilitas [Tidak Stres, Stres]: [0.3734007 0.6265993] -> Prediksi: 1
Data ke-4 -> Probabilitas [Tidak Stres, Stres]: [0.3000113 0.6999887] -> Prediksi: 1
Data ke-5 -> Probabilitas [Tidak Stres, Stres]: [0.8760007 0.1239993] -> Prediksi: 0
```

5. Analisis dan Jawaban Pertanyaan Evaluasi

1. Mengapa model Regresi Logistik cocok untuk dataset ini?

- Jawaban: Karena target yang mau kita tebak cuma ada dua kemungkinan (klasifikasi biner), yaitu mahasiswa Stres (1) atau Tidak Stres (0). Regresi Logistik sangat pas untuk tipe data ini karena dia bisa mengubah hubungan variabel angka seperti jumlah tugas dan jam tidur menjadi skor probabilitas antara 0 sampai 1 untuk menentukan keputusan akhir. Hasil akurasi di Colab yang mencapai 93.75% membuktikan kalau model ini memang sangat akurat memetakan data survei kita.

2. Mengapa nilai probabilitas dari `predict_proba()` penting?

- Jawaban: Nilai probabilitas ini penting karena kita bisa tahu seberapa yakin atau mantap model AI dalam mengambil keputusan, bukan cuma sekedar tebak-tebakan hitam di atas putih. Contohnya bisa dilihat langsung pada output data uji ke-1 dan ke-2 di atas:
 - Pada Data ke 1, model sangat yakin mahasiswa tidak stres karena peluang Tidak Stres nya tinggi banget di angka 98.5%.
 - Pada Data ke 2, model mendeteksi kondisi kritis karena peluang Stres mahasiswa tersebut mencapai 98.3%. Dengan angka probabilitas ini, kita bisa menyortir mana mahasiswa yang tingkat stresnya sudah di tahap waspada atau masih aman.

3. Faktor apa yang paling berpengaruh terhadap status stres mahasiswa?

- Jawaban: Jika melihat tren dari data survei, faktor yang paling sensitif bikin mahasiswa stres adalah banyaknya jumlah tugas yang melewati deadline dan kurangnya rata-rata jam tidur. Tugas yang menumpuk ditambah dikejar-kejar batas waktu bikin beban pikiran langsung naik drastis, lalu diperparah lagi dengan waktu istirahat (jam tidur) yang kepotong. Kombinasi dua hal ini yang paling kuat memicu status stres di model AI kita.

4. Apa kelemahan model ini jika hanya mengandalkan fitur dari dataset?

- Jawaban: Kelemahannya adalah model ini terlalu menyederhanakan masalah stres mahasiswa. Model menganggap pemicu stres itu cuma urusan kuliah seperti jumlah tugas dan jam tidur saja. Padahal di kehidupan nyata, penyebab stres mahasiswa itu kompleks banget dan banyak faktor eksternal lain yang tidak terekam di spreadsheet kita, misalnya masalah keuangan (finansial), konflik personal dengan keluarga atau teman, masalah asmara, hingga kondisi kesehatan fisik masing-masing mahasiswa.